Motor de búsqueda de funciones similares vía AST

Lenguajes Formales y Autómatas · UCSE · 2026 · Integrantes: Rossi, Ignacio · Sara, Nicolas · Sarra, Juan Pablo

1. Enfoque

`ast.parse` aplica la gramática de Python a la cadena de entrada. Hasta Python 3.8 lo hacía un parser **LL(1)** que construía el árbol de derivación concreto (CST) y luego lo convertía en AST; desde 3.9 lo hace un parser PEG descendente recursivo, con backtracking y memoización (PEP 617). El AST es una versión abstracta del árbol de derivación: descarta la puntuación y colapsa las cadenas de producciones del tipo `expr → … → atom`, de forma análoga a eliminar reglas de redenominación `U ::= V`. Comparar funciones sobre ese árbol en vez de sobre el texto las vuelve independientes del formato, los comentarios y los nombres. La herramienta sigue cinco etapas, cada una en su módulo: **extracción → normalización → serialización pre-order → métrica → búsqueda top-K**.

2. Decisiones de diseño

Extracción (`extraccion.py`). Recorre el directorio con `os.walk`, salteando directorios ocultos y de entornos virtuales. Lee cada `.py` con `tokenize.open`, que respeta la declaración de codificación (PEP 263), y extrae todos los `FunctionDef` y `AsyncFunctionDef`, incluidos métodos y funciones anidadas. De cada una guarda nombre, nombre completo (`Clase.metodo`), archivo, línea, subárbol y código. Los archivos que no parsean (por ejemplo, Python 2) se informan y se omiten.

Normalización (`normalizacion.py`, un `ast.NodeTransformer` sobre una copia del subárbol):

- *Variables y parámetros* → `VAR1, VAR2, …` en orden de primera aparición, que coincide con el orden de la serialización. Cada identificador recibe siempre el mismo token, así se preserva la correspondencia. Lo mismo vale para los nombres ligados por `def`/`class` anidados, `except … as`, `import … as`, `global`, `match` y los nombres libres (funciones o módulos externos).
- *Builtins* (`len`, `range`, `ValueError`…) se conservan, salvo que la función los redefina: son vocabulario del lenguaje, y cambiar `len` por `sum` cambia el significado.
- *Atributos y métodos* (`.append`, `.items`) se conservan porque describen la operación usada. La excepción es `self.<nombre_función>(…)`, que es recursión y pasa a `FUNC`. La calibración mostró que mapear cualquier atributo con el nombre de la función era un error: en `close()`, la llamada `self.sock.close()` es delegación a otro objeto.
- *Literales* numéricos, strings y bytes → `LIT`. Se conservan `None`, `True` y `False`, porque son palabras reservadas y no datos: `return None` y `return False` no son equivalentes.
- *Nombre de la función* → `FUNC`, también en las llamadas recursivas, para que no influya en la comparación.
- Se eliminan docstrings y strings sueltos, anotaciones de tipo y decoradores de la función; `x: int = 0` se reescribe como `x = 0`.

Serialización. Es un recorrido pre-order propio sobre `ast.iter_fields`. No usa `ast.walk` porque su orden no está especificado (en CPython es por niveles). Emite un token por nodo con su tipo y su detalle normalizado (`For`, `Name:VAR2`, `Constant:LIT`, `Attribute:append`) y omite los contextos `Load`/`Store`, que son redundantes. El pre-order puro **no es inyectivo**: `for x in xs: a(); b()` y `for x in xs: a()` seguido de `b()` producen la misma secuencia. Por eso cada bloque de sentencias se encierra entre `{ }`, lo que preserva el anidamiento. Por ejemplo, `def f(a): return a + 1` produce `FunctionDef:FUNC arguments arg:VAR1 { Return BinOp Name:VAR1 Add Constant:LIT }`.

3. Métrica elegida y umbral

Se implementan dos métricas, más su promedio (`combinada`):

- **Jaccard sobre n-gramas** de tokens, como multiconjunto (`Counter`, donde `&` es la intersección y `|` la unión). Así se tiene en cuenta cuántas veces se repite cada patrón.
- **Levenshtein normalizada**: `1 − d/max(|a|,|b|)`.

Una decisión clave: los n-gramas se calculan sobre la secuencia con `VARn → VAR`. El índice de `VARn` es información *global*: agregar al comienzo una variable renumera todas las demás y cambia casi todos los n-gramas, aunque la estructura local sea la misma. Con índices, el recall ante esa mutación era 0.40; sin índices, pasa a 1.00. La correspondencia entre variables la sigue aprovechando Levenshtein, que alinea la secuencia completa.

Calibración (`calibracion/calibrar.py`, reproducible). El corpus son 6924 funciones de la biblioteca estándar de Python. Sobre 400 de ellas (≥ 30 tokens) se generan clones sintéticos de tipo 2 (renombrado consistente y cambio de literales) y de tipo 3 (intercambiar o mover sentencias, insertar, borrar y combinaciones). Los negativos son 54 826 pares de funciones distintas *del mismo archivo*: mismo autor y estilo, el caso más difícil. Como el umbral se fija para tener a lo sumo 1% de falsos positivos sobre esos pares, se obtiene:

Métrica	Umbral (FPR ≤ 1%)	Recall clones tipo 3	AUC	Búsqueda: original en el top-1	Seg/consulta
Jaccard n=1	0.78	0.958	0.992	–	–
Jaccard n=2	0.67	0.971	0.996	0.989	0.12
Jaccard n=3	**0.59**	**0.974**	**0.997**	**0.989**	**0.18**
Jaccard n=4	0.53	0.970	0.997	0.989	0.20
Jaccard n=6	0.44	0.943	0.996	–	–
Levenshtein	0.66	0.835	0.988	0.900	2.99
Combinada (n=3)	0.63	0.938	0.995	0.989	3.58

En la columna de búsqueda, cada uno de 90 clones se busca entre las 6924 funciones del corpus y se cuenta en qué proporción de los casos el original sale primero.

Se elige **Jaccard con n = 3** como métrica por defecto, porque tiene el mayor recall y el mejor AUC, y en la búsqueda iguala a la combinada siendo 20 veces más rápida. Con n = 1 o 2 los n-gramas son demasiado genéricos: la mediana de los negativos sube de 0.07 a 0.34 con n = 1, que es una "bolsa de tipos de nodo" y no ve estructura. Con n ≥ 4 cada edición rompe más n-gramas y el recall cae ante cambios combinados. Un 3-grama equivale a un camino de tres nodos consecutivos del pre-order, por ejemplo `For Name:VAR Call`. Jaccard además tolera los reordenamientos, mientras que Levenshtein los paga como un borrado más una inserción. El **umbral de "candidata a duplicado" es 0.60** (0.59 redondeado hacia arriba); Levenshtein usa 0.66 y la combinada 0.63, todos configurables con `--umbral`. Todas las renombradas tipo 2 dieron exactamente 1.0 (398/400; las 2 restantes son artefactos de la mutación). Las cifras son pesimistas, porque entre los "negativos" hay clones reales de la stdlib.

Búsqueda. La comparación es todos contra todos, pero con una **poda exacta**: el score de Jaccard está acotado por `min(|A|,|B|)/max(|A|,|B|)` y el de Levenshtein por el cociente de las longitudes. Los candidatos se recorren de mayor a menor cota y la búsqueda se detiene cuando la cota no supera al K-ésimo mejor score. Un test verifica que el resultado coincide con el de la fuerza bruta.

Validación. La suite de `pytest` tiene 81 tests y pasa completa en Python 3.11; en 3.9, el único test que usa `match` se saltea. Hay casos positivos (variables renombradas, líneas reordenadas, sentencia agregada, recursión renombrada) y negativos (cálculo numérico contra procesamiento de texto, recursión contra ordenamiento, búsqueda binaria contra conteo). También hay tests para cada regla de normalización y para la CLI, y tests que documentan las limitaciones de la sección 4.

4. Limitaciones conocidas: qué clones NO detecta

- **Clones semánticos (tipo 4)**: un bucle acumulador y `sum(xs)/len(xs)` hacen lo mismo con árboles distintos. Tampoco detecta equivalencias como `for` ↔ `while`, bucle ↔ comprensión, `if/else` ↔ ternario, `a+b` ↔ `b+a` o `not a<b` ↔ `a>=b`.
- **Clones parciales**: un fragmento copiado dentro de una función más grande, o una función partida en dos (*extract method*). Como se compara función contra función, el parecido se diluye.
- **Funciones chicas**: pocas ediciones mueven mucho el score. Agregar dos sentencias a una función de 29 tokens deja el Jaccard en 0.59, debajo del umbral (hay un test que lo documenta). Además, las funciones triviales se parecen todas entre sí, por eso existe `--min-tokens` en el reporte de duplicados.
- **Levenshtein sola** penaliza los reordenamientos: el clon reordenado de `varianza` da 0.50 con Levenshtein y 0.76 con Jaccard. **Jaccard**, a la inversa, ignora el orden más allá de 3 tokens, así que dos funciones con los mismos fragmentos en distinto orden puntúan alto.
- **Renombrar atributos** (`self.total` → `self.suma`) baja algo el score, porque los nombres de atributos se conservan.
- Una función externa incluye los tokens de sus funciones anidadas. Los archivos que `ast.parse` rechaza (Python 2) no se analizan.

5. Posibles mejoras

- **Numeración canónica de `VARn`**: asignar los números recorriendo cada bloque con sus sentencias ordenadas por su forma anónima, para que reordenar no renumere. Un prototipo subió el recall de Levenshtein de 0.831 a 0.867.
- **Fingerprints con *winnowing*** (el algoritmo de MOSS) para detectar clones parciales dentro de funciones, y **MinHash/LSH** para escalar a miles de archivos sin comparar todos contra todos.
- **Distancia de edición entre árboles** (Zhang–Shasha) en lugar de entre secuencias.
- **Más normalizaciones**: `while` ↔ `for`, comprensiones ↔ bucles, orden canónico de operandos conmutativos.
- **Ponderar los n-gramas por rareza** (TF-IDF), para que los patrones genéricos como `Assign Name:VAR Constant:LIT` pesen menos.
- Un **grafo de dependencias de datos** (PDG), para acercarse a los clones semánticos.